

Lecture 7 – SQL Part 1

CPE241 – Database Systems

3 March 2026

Dr. Jaturon Harnsomburana

Dr. Piyanit Ua-areemitr

Department of Computer Engineering

King Mongkut's University of Technology Thonburi



Recap

- SQL - standard language that can be interact with structured data on databases.
- Categories of SQL commands
 - Data Definition Language (DDL)
 - Data Manipulation Language (DML)
 - Data Query Language (DQL)
 - Data Control Language (DCL)
 - Transaction Control Language (TCL)

Today's Goals

- Data Definition Language (DDL)
- Data Manipulation Language (DML)
- Data Query Language (DQL)

DDL

Command	Description	Syntax
CREATE	Create <i>database or its objects</i> (table, index, function, views, store procedure, and triggers)	CREATE TABLE table_name (column1 data_type, column2 data_type, ...);
DROP	Delete objects from the database	DROP TABLE table_name;
ALTER	Alter the structure of the database	ALTER TABLE table_name ADD COLUMN column_name data_type;
TRUNCATE	Remove all records from a table, including all spaces allocated for the records are removed	TRUNCATE TABLE table_name;
COMMENT	Add comments to the data dictionary	COMMENT 'comment_text' ON TABLE table_name;
RENAME	Rename an object existing in the database	RENAME TABLE old_table_name TO new_table_name;

Source: <https://www.geeksforgeeks.org/sql-ddl-dql-dml-dcl-tcl-commands/>

DML

Command	Description	Syntax
INSERT	Insert data into a table	INSERT INTO table_name (column1, column2, ...) VALUES (value1, value2, ...);
UPDATE	Update existing data within a table	UPDATE table_name SET column1 = value1, column2 = value2 WHERE condition;
DELETE	Delete records from a database table	DELETE FROM table_name WHERE condition;
LOCK	Table control concurrency	LOCK TABLE table_name IN lock_mode;
CALL	Call a PL/SQL or JAVA subprogram	CALL procedure_name(arguments);
EXPLAIN PLAN	Describe the access path to data	EXPLAIN PLAN FOR SELECT * FROM table_name;

Source: <https://www.geeksforgeeks.org/sql-ddl-dql-dml-dcl-tcl-commands/>

Check Point

- Exam problem

Example database

ENROLLMENT

sid	cid	grade
53666	Jazz101	C
53666	Reggae203	B
53650	Topology112	A
53666	History105	B

STUDENT

sid	name	login	age	gpa
53666	Jones	jones@cs	18	3.4
53688	Smith	smith@eecs	18	3.2
53650	Smith	smith@math	19	3.8

DML

ENROLLMENT

sid	cid	grade
53666	Jazz101	C
53666	Reggae203	B
53650	Topology112	A
53666	History105	B

STUDENT

sid	name	login	age	gpa
53666	Jones	jones@cs	18	3.4
53688	Smith	smith@eecs	18	3.2
53650	Smith	smith@math	19	3.8

- **UPDATE:** Modifies existing records.

```
UPDATE table_name SET column1_name = value1, ... WHERE condition;
```

```
UPDATE student SET gpa = 3.0 WHERE name = 'Smith';
```

- **DELETE:** Removes records.

```
DELETE FROM table_name WHERE condition;
```

```
DELETE FROM enrollment WHERE cid = 'Reggae';
```


DQL

Command	Description	Syntax
SELECT	It is used to retrieve data from the database	SELECT column1, column2, ...FROM table_name;

Source: <https://www.geeksforgeeks.org/sql-ddl-dql-dml-dcl-tcl-commands/>

DQL main objective: retrieves data from one or more tables.

```
SELECT {ColumnName(s)}  
  
FROM {TableName(s)}  
  
WHERE {Conditions(s)};
```

SELECT * WHERE

```
SELECT sid, cid FROM enrollment;
```

Result: 4 rows

```
SELECT * FROM enrollment;
```

Result: 4 rows

```
SELECT * FROM enrollment WHERE grade = 'B';
```

Result: 2 rows

ENROLLMENT

sid	cid	grade
53666	Jazz101	C
53666	Reggae203	B
53650	Topology112	A
53666	History105	B

DISTINCT

```
SELECT sid, grade FROM enrollment;
```

Result: 4 rows

```
SELECT DISTINCT sid, grade FROM enrollment;
```

Result: 3 rows

ENROLLMENT

sid	cid	grade
53666	Jazz101	C
53666	Reggae203	B
53650	Topology112	A
53666	History105	B

sid	grade
53666	C
53666	B
53650	A
53666	B

sid	grade
53666	C
53666	B
53650	A

WHERE Clause – AND OR

```
SELECT *  
FROM enrollment  
WHERE sid = '53666' AND grade = 'C';  
Result: 1 row
```

```
SELECT *  
FROM enrollment  
WHERE sid = '53650' OR grade = 'B';  
Result: 3 rows
```

ENROLLMENT

sid	cid	grade
53666	Jazz101	C
53666	Reggae203	B
53650	Topology112	A
53666	History105	B

sid	cid	grade
53666	Jazz101	C

sid	cid	grade
53666	Reggae203	B
53650	Topology112	A
53666	History105	B

WHERE Clause – IN

```
SELECT *  
FROM enrollment  
WHERE grade IN ('A', 'C');  
Result: 2 rows
```

```
SELECT *  
FROM enrollment  
WHERE grade NOT IN ('A');  
Result: 3 rows
```

ENROLLMENT

sid	cid	grade
53666	Jazz101	C
53666	Reggae203	B
53650	Topology112	A
53666	History105	B

sid	cid	grade
53666	Jazz101	C
53650	Topology112	A

sid	cid	grade
53666	Jazz101	C
53666	Reggae203	B
53666	History105	B

WHERE Clause – BETWEEN

ENROLLMENT

sid	cid	grade
53666	Jazz101	C
53666	Reggae203	B
53650	Topology112	A
53666	History105	B

```
SELECT *  
FROM enrollment  
WHERE grade BETWEEN 'A' AND 'B';  
Result: 3 rows
```

sid	cid	grade
53666	Reggae203	B
53650	Topology112	A
53666	History105	B

WHERE Clause – Math Symbols

ENROLLMENT

sid	cid	grade
53666	Jazz101	C
53666	Reggae203	B
53650	Topology112	A
53666	History105	B

```
SELECT *  
FROM enrollment  
WHERE grade >= 'B' AND grade <= 'C';  
Result: 3 rows
```

sid	cid	grade
53666	Jazz101	C
53666	Reggae203	B
53666	History105	B

WHERE Clause – LIKE

- The SQL keyword LIKE used for string pattern matching and can be combined with wildcard symbols:
 - SQL 92 Standard (SQL Server, MySQL, etc.)

_ : exactly one character
% : any set of one or more characters

- Microsoft Access (based on MS DOS)

? : exactly one character
* : any set of one or more characters

WHERE Clause – LIKE

STUDENT

sid	name	login	age	gpa
53666	Jones	jones@cs	18	3.4
53688	Smith	smith@eecs	18	3.2
53650	Smith	smith@math	19	3.8

```
SELECT *  
FROM student  
WHERE name LIKE 'Smith';
```

Result: 2 rows

sid	name	login	age	gpa
53688	Smith	smith@eecs	18	3.2
53650	Smith	smith@math	18	3.8

WHERE Clause – LIKE

```
SELECT *  
FROM enrollment  
WHERE cid LIKE '%10%';  
Result: 2 rows
```

```
SELECT *  
FROM enrollment  
WHERE cid LIKE '%1__';  
Result: 3 rows
```

ENROLLMENT

sid	cid	grade
53666	Jazz101	C
53666	Reggae203	B
53650	Topology112	A
53666	History105	B

sid	cid	grade
53666	Jazz101	C
53666	History105	B

sid	cid	grade
53666	Jazz101	C
53650	Topology112	A
53666	History105	B

ORDER BY

```
SELECT * FROM enrollment ORDER BY grade;
```

sid	cid	grade
53650	Topology112	A
53666	Reggae203	B
53666	History105	B
53666	Jazz101	C

```
SELECT * FROM enrollment ORDER BY sid DESC, cid ASC;
```

sid	cid	grade
53666	History105	B
53666	Jazz101	C
53666	Reggae203	B
53650	Topology112	A

ENROLLMENT

sid	cid	grade
53666	Jazz101	C
53666	Reggae203	B
53650	Topology112	A
53666	History105	B

Built-in Aggregate Functions

```
SELECT AVG(gpa)
FROM student;
```

Rename by AS:

```
SELECT AVG(gpa) AS AverageGPA
FROM student;
```

STUDENT

sid	name	login	age	gpa
53666	Jones	jones@cs	18	3.4
53688	Smith	smith@eecs	18	3.2
53650	Smith	smith@math	19	3.8

averagegpa

3.4666666984558105

Built-in Aggregate Functions

```
SELECT AVG(gpa) AS AverageGPA,  
       MIN(gpa) AS MinGPA,  
       MAX(gpa) AS MaxGPA  
FROM student;
```

```
SELECT COUNT(*) AS NumberOfRows  
FROM student;
```

```
SELECT SUM(age) AS CumulativeAge  
FROM student;
```

STUDENT

sid	name	login	age	gpa
53666	Jones	jones@cs	18	3.4
53688	Smith	smith@eecs	18	3.2
53650	Smith	smith@math	19	3.8

averagegpa	mingpa	maxgpa
3.4666666984558105	3.2	3.8

numberofrows
3

xxx
yyy

JOIN

```
SELECT name, cid, grade
FROM enrollment JOIN student
ON enrollment.sid = student.sid;
```

Results: 4 rows

```
SELECT name, cid, grade
FROM enrollment, student
WHERE enrollment.sid = student.sid;
```

Results: 4 rows

ENROLLMENT

sid	cid	grade
53666	Jazz101	C
53666	Reggae203	B
53650	Topology112	A
53666	History105	B

STUDENT

sid	name	login	age	gpa
53666	Jones	jones@cs	18	3.4
53688	Smith	smith@eecs	18	3.2
53650	Smith	smith@math	19	3.8

name	cid	grade
Jones	Jazz101	C
Jones	Reggae203	B
Smith	Topology112	A
Jones	History105	B

JOIN Types

- **INNER JOIN** is the default type of join in a joint table.
 - Rows is included in the result only if a matching row is existing in the other relation.
- **OUTER JOIN**
 - **LEFT OUTER JOIN** - every rows in the left table must appear in the joint table.
 - **RIGHT OUTER JOIN** - every rows in the right table must appear in the joint table.
 - **FULL OUTER JOIN** - combines the results of both left and right outer joins.
- If there is no matching row in the other relation, padded with NULL values for attributes of the other table.

JOIN Types

```
SELECT * FROM Table1 ..... JOIN Table 2 ON Table1.Name1 = Table2.Name2;
```

Table1

Name1
Alice
Bob
Carol
David

Table2

Name2
Carol
David
Elsa
Frank

INNER

Name1	Name2
Carol	Carol
David	David

LEFT OUTER

Name1	Name2
Alice	NULL
Bob	NULL
Carol	Carol
David	David

RIGHT OUTER

Name1	Name2
Carol	Carol
David	David
NULL	Elsa
NULL	Frank

FULL OUTER

Name1	Name2
Alice	NULL
Bob	NULL
Carol	Carol
David	David
NULL	Elsa
NULL	Frank

Aliasing and Renaming

- Ambiguous attribute names: a same name can be used for two (or more) attributes in *different relations*.
- Must qualify the attribute name with the relation's name to prevent ambiguity.
- Declare alternative relation names

Aliasing and Renaming

- Schema:
 - EMPLOYEE (Fname, Minit, Lname, Ssn, Bdate, Address, Sex, Salary, Super_ssn, Dno)

For each employee, retrieve the employee's first and last name and the first and last name of his or her immediate supervisor.

```
SELECT E.Fname, E.Lname, S.Fname, S.Lname
FROM EMPLOYEE AS E, EMPLOYEE AS S
WHERE E.Super_ssn = S.Ssn;
```

Wrap up

- DML: UPDATE, DELETE
- DQL: SELECT
 - DISTINCT
 - WHERE Clause
 - ORDER BY
 - Build-in aggregate functions
 - JOIN